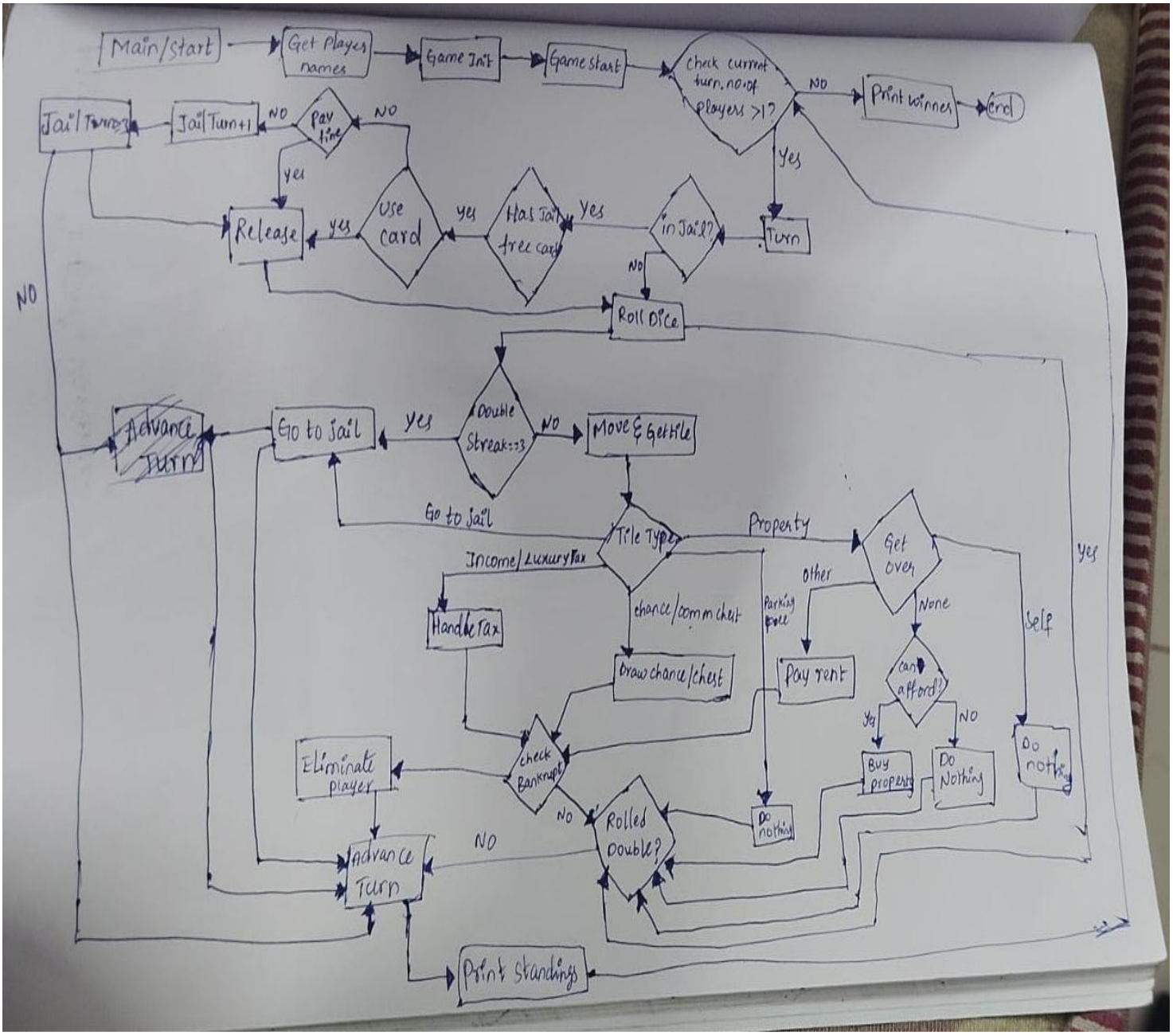
White Box Testing Report

1.1 Control Flow Graph



1.2 Code Quality Analysis

Using pylint, we identified several code quality issues:

- Missing module, class, and function docstrings across all `.py` files.
- Unused variables and imports (e.g., `math` in bank.py, `sys` in player.py).
- Superfluous parenthesis and improper `True` singletons comparisons.

- Broad `Exception` catching in `ui.py`.

Changes made:

- Iteration 1: Added missing docstrings to modules, classes, and functions.
- Iteration 2: Removed unused imports and unused variables.
- Iteration 3: Fixed syntax issues like singletons comparisons.
- Iteration 4: Initialized missing attributes and resolved bare exceptions in ui.py.
- Iteration 5: Refactored `Player` jail state into a `JailState` dataclass to natively resolve `too-many-instance-attributes` warnings without suppressions.
- Iteration 6: Extracted `Decks`, `GameEngine`, and dictionary-based tile handlers in `Game` to cleanly resolve `too-many-branches` and `too-many-instance-attributes`.
- Iteration 7: Refactored `Property` initialization via a `PropertyConfig` dataclass and a `create_property()` factory function to resolve `too-many-arguments`, `too-many-positional-arguments`, and `too-many-instance-attributes`.

1.3 White Box Test Cases

Bank Logic Errors

- **Test Case**: `test_bank_give_loan_deducts_funds`
- **Why it's needed**: Covers the `give_loan` branch to ensure bank funds are successfully reduced when issuing a loan. A logical edge case where the system generates free money.
- **Errors found**: The `bank.give_loan()` method added money to the player but failed to deduct the funds from `self._funds`! This caused free money to enter the economy.
- **Fix**: Added `self._funds -= amount` to `give_loan`.
- **Test Case**: `test_bank_payout_insufficient_funds`

- **Why it's needed**: Ensures the `pay_out` function correctly handles the boundary case of paying more than available funds.
- **Errors found**: No error, behaved as expected by throwing ValueError.

Player Logic Errors

- **Test Case**: `test_player_net_worth_includes_properties`
- **Why it's needed**: Checks the calculation of player net worth to verify it factors in the player's properties, rather than just raw cash.
- **Errors found**: Th `net_worth` function simply returned `self.balance`, ignoring the value of properties the player holds.
- **Fix**: Updated `net_worth` to iterate over properties and add their price (or mortgaged value) to the total.
- **Test Case**: `test_player_move_passes_go`
- **Why it's needed**: Verifies the board boundary (pass GO) wrapping behavior and ensures the `GO_SALARY` is correctly awarded.
- **Errors found**: The code only awarded the salary if the player landed EXACTLY on position `0` (`if self.position == 0:`), missing out if they overshot.
- **Fix**: Saved `old_position` and changed condition to `if old_position + steps >= BOARD_SIZE`.

Game Logic Errors

- **Test Case**: `test_find_winner_returns_highest_net_worth`
- **Why it's needed**: Verifies that the end condition properly chooses the player with the highest net worth.
- **Errors found**: The `find_winner` function mistakenly used Python's `min()` function, causing the player with the *lowest* net worth to be declared the winner.
- **Fix**: Changed `min()` to `max()`.
- **Test Case**: `test_trade_adds_money_to_seller`
- **Why it's needed**: Validates the property trading interaction code.
- **Errors found**: During a trade, the cash was deducted from the buyer, but never added to the seller's balance!
- **Fix**: Added `seller.add_money(cash_amount)` to the `trade` method.